Week 4 - AWS Control Tower (landing zone, controls, Account Factory, AFT, customizations)

DOP-C02 Domains 2 and 6. Typical questions:

1. Set up a **secure multi-account environment quickly** with AWS best practices = **Control Tower landing zone**.
2. **Prevent / detect / block before provisioning** non-compliant resources across OUs = **preventive / detective / proactive controls**.
3. Self-service **account vending** with a baseline = **Account Factory** or **AFT** (Terraform GitOps).
4. **Automatically apply custom CloudFormation and SCPs to every new account** = **CfCT**, **AFC blueprints** or **lifecycle events**.
5. An account was moved / an SCP edited and the dashboard shows **drift** = **repair / re-register**.

The big idea: Control Tower is an **orchestration layer** that **sets up and governs** Organizations, IAM Identity Center, CloudTrail, Config, Service Catalog, CloudFormation StackSets, S3 and Security Hub. **No extra charge** for Control Tower; you pay for the underlying services (mostly Config, CloudTrail, S3).

Part 1 - The landing zone

```
Organization (management account = where Control Tower runs, home Region)
  Root
    Security OU  (foundational, created by CT)
      Log Archive account  -> central S3 buckets: CloudTrail + Config logs (+ access logs)
      Audit account        -> security team roles, SNS notifications, Config aggregator
    Sandbox OU   (optional additional OU)
    your OUs     (Workloads, Infrastructure...) -> "registered" to bring under CT governance
```

Item	Detail
Home Region	Chosen at setup, **can't be changed**; CT resources live here
Governed Regions	Where CT applies controls and Config; add more later; optional **Region deny control** blocks **ungoverned Regions** (except global services)
Shared accounts	**Log Archive** (immutable logs) and **Audit** (security tooling, recommended **delegated admin** for GuardDuty / Security Hub); new or existing accounts
Organizations	Security + optional Sandbox OUs, **mandatory controls** (SCPs)
CloudTrail	**Organization trail** to the Log Archive bucket (or your own trail)
AWS Config	Recorder + delivery channel in every governed account / Region; **aggregator** in Audit
IAM Identity Center	SSO directory, groups, permission sets (AWSControlTowerAdmins, AWSSecurityAuditors), or **self-managed identity**
Notifications	Aggregated **SNS topics in Audit** (compliance changes, security notifications)
KMS / retention	Optional CMK for CloudTrail / Config; configurable log lifecycle

Landing zone lifecycle

- **Versions**: **Update landing zone** (console or `UpdateLandingZone` with a manifest), then **re-register OUs / update accounts**.
- **APIs / IaC**: `CreateLandingZone`, `UpdateLandingZone`, `ResetLandingZone`; CloudFormation `AWS::ControlTower::LandingZone`, `AWS::ControlTower::EnabledControl`, `AWS::ControlTower::EnabledBaseline`.
- **Existing organization**: set up CT, then **register existing OUs** and **enroll existing accounts**.
- **Don't modify CT-managed resources**: `AWSControlTower*` stacks / StackSets, `aws-guardrails-*` SCPs, `AWSControlTowerExecution` role, log buckets - causes **drift**.

Part 2 - Governing OUs and accounts

Action	What it does	Notes
Register OU	Applies the CT **baseline** (Config, CloudTrail, roles, mandatory controls) to **all accounts in an existing OU**	Nested OUs supported; precheck first
Enroll account	Brings **one existing account** under CT	Needs `AWSControlTowerExecution` **role** (trusts management account) and no conflicting Config recorder / delivery channel
Re-register OU	Re-applies the baseline (after landing zone update or drift)	
Update account	Re-applies the baseline to one account (Account Factory)	
Move outside CT	Moving accounts in Organizations directly	Causes **drift**

Enrollment failures: **missing** `AWSControlTowerExecution` **role**, **existing Config recorder / delivery channel**, **SCPs blocking CT actions**.

Part 3 - Controls (formerly guardrails)

Behaviour

Behaviour	Implemented with	When	Example
Preventive	**SCPs** (and **RCPs**, some declarative policies)	**Blocks** the API call	Disallow CloudTrail changes, disallow log archive deletion, **Region deny**, disallow root access keys
Detective	**AWS Config rules** (and Security Hub controls)	**Detects** after the fact, reports non-compliance	Detect root MFA, detect public S3 read, detect unencrypted EBS
Proactive	**CloudFormation Hooks**	**Checks resources before CloudFormation provisions them**; fails / warns	Require S3 encryption config in templates, require Lambda in a VPC

Guidance

Guidance	Meaning
Mandatory	**Always enabled** on registered OUs, **can't be disabled** - mostly protect the landing zone
Strongly recommended	Best practices (disallow root access keys, detect MFA, disallow public S3)
Elective	Optional (disallow S3 encryption changes, disallow deleting bucket logging, data residency)

Key facts

- **Controls are enabled per registered OU** and apply to all its accounts (nested too): console, `EnableControl` / `DisableControl` (control ARN + OU ARN), `AWS::ControlTower::EnabledControl`.
- Detective results on the **CT dashboard**, aggregated in **Audit** (aggregator, SNS).
- **Control catalog** maps to frameworks (NIST, PCI, CIS); includes Security Hub controls via the **Service-Managed Standard: AWS Control Tower**.
- **Data residency controls** + **Region deny** (landing zone or per OU).
- Preventive (SCP) controls **don't affect the management account**; CT roles are exempted.
- **Detective controls only report** - auto-fix needs **Config remediation (SSM Automation)** or EventBridge + Lambda.

"Stop it" = **preventive**; "find what's wrong" = **detective**; "stop non-compliant resources in CloudFormation templates before creation" = **proactive**.

Part 4 - Account Factory

Account Factory (console / Service Catalog)

- **Provision accounts** with the CT baseline (Config, CloudTrail, roles, OU controls, Identity Center access).
- Inputs: email, display name, **target OU**, SSO user.
- It's the **Service Catalog product "AWS Control Tower Account Factory"**: automate with `ProvisionProduct` (or CT account APIs), offer **self-service** through Service Catalog.
- **Network configuration**: VPC for new accounts (create or not, CIDR, public / private subnets, Regions) or **no VPC / delete default VPC**.
- **Update / enroll** existing accounts here too.

Account Factory Customization (AFC) - blueprints

- **Blueprints** = **Service Catalog products** (CloudFormation or Terraform) in a **hub account**.
- **Applied automatically when an account is provisioned or updated** through Account Factory (choose in the console); partner blueprints available.
- **Simplest built-in, console-driven customization** - no pipeline.

Part 5 - Account Factory for Terraform (AFT)

Terraform **GitOps pipeline** for **account vending + customizations**, deployed with a **Terraform module** into a **dedicated AFT management account**.

```
Git repos (CodeCommit / GitHub / GitLab / Bitbucket / GitHub Enterprise)
  aft-account-request                    -> one module call per account (email, OU, SSO, tags)
  aft-global-customizations              -> Terraform for EVERY AFT account
  aft-account-customizations             -> folders applied to accounts that reference them
  aft-account-provisioning-customizations -> Step Functions before customizations (ITSM / CMDB)
        | git push
        v
AFT account: CodePipeline / CodeBuild, Step Functions, Lambda, DynamoDB, SQS, Terraform
        |
        v
Control Tower Account Factory creates / updates the account -> AFT runs customizations in it
```

Feature	Detail
Account request	Commit a Terraform file -> AFT calls **Account Factory** -> account in target OU
Customizations	**Global** + **account-specific** Terraform, plus pre / post API helper scripts (Python / Bash)
Provisioning customizations	Step Functions workflow (for example register in ServiceNow)
Feature options	**Delete default VPCs**, **CloudTrail data events**, **Enterprise Support enrollment**
Re-run customizations	`aft-invoke-customizations` Step Functions (all, by OU, or selected accounts)
Terraform	OSS (state in S3 + DynamoDB), HCP Terraform, Terraform Enterprise
Prerequisites	**CT landing zone** + **dedicated AFT management account** (own OU)

"Team uses **Terraform** and wants **GitOps** account vending" = **AFT**. CloudFormation = **CfCT / AFC / StackSets**.

Part 6 - Customizations for AWS Control Tower (CfCT)

AWS solution (CloudFormation in the **management account**) that applies **your CloudFormation templates (via StackSets) and SCPs / RCPs** to OUs and accounts, GitOps-style, **automatically for new accounts**.

```
Configuration package (CodeCommit or S3):
  manifest.yaml
  templates/*.yaml   (CloudFormation)
  policies/*.json    (SCPs / RCPs)
      | change -> CodePipeline (CodeBuild validate -> StackSets / Organizations API)
      v
Deploy to OUs / accounts / Regions in manifest.yaml
Also triggered by lifecycle event CreateManagedAccount -> new accounts get customizations
```

```
region: us-east-1
version: 2021-03-15
resources:
  - name: baseline-iam-roles
    resource_file: templates/iam-roles.yaml
    deploy_method: stack_set
    deployment_targets:
      organizational_units: [Workloads, Sandbox]
    regions: [us-east-1]
  - name: deny-leave-org
    resource_file: policies/deny-leave.json
    deploy_method: scp
    deployment_targets:
      organizational_units: [Workloads]
```

- `deploy_method`: `stack_set`, `scp`, `rcp`; parameters from SSM or other stack outputs.
- "New Account Factory accounts automatically get our **CloudFormation baseline + custom SCPs**, as code in a pipeline" = **CfCT**.

Part 7 - Lifecycle events

Emitted through CloudTrail to **EventBridge** in the **management account, home Region**, when operations complete:

Event	When
`CreateManagedAccount`	Account Factory finished creating **and enrolling** an account
`UpdateManagedAccount`	Account updated / re-enrolled
`EnableGuardrail` / `DisableGuardrail`	Control enabled / disabled
`SetupLandingZone` / `UpdateLandingZone`	Landing zone set up / updated
`RegisterOrganizationalUnit` / `DeregisterOrganizationalUnit` / `PrecheckOrganizationalUnit`	OU governance

- EventBridge on `CreateManagedAccount` -> Lambda / Step Functions / SSM Automation (GuardDuty membership, budgets, IPAM, CMDB, notify).
- `serviceEventDetails` contains account ID, name, OU, status.
- Use CT lifecycle events, **not Organizations** `CreateAccount` - they fire **after the baseline is in place**.

Part 8 - Choosing a customization method

Need	Choose
Simple, console-driven template at account create / update	**AFC blueprints**
CloudFormation + SCPs / RCPs as code, auto-applied to new accounts	**CfCT**
Terraform GitOps vending + customizations	**AFT**
Large regulated, config-driven environments (works with CT)	**Landing Zone Accelerator on AWS (LZA)**
One-off reaction to account creation	`CreateManagedAccount` **-> EventBridge -> Lambda / SSM**
Stack to all accounts in OUs (with or without CT)	**StackSets (service-managed, auto-deployment)**

Part 9 - Drift and integrations

Drift type	Example	Fix
Account moved	Enrolled account moved to another / unregistered OU via Organizations	Move back, **update / re-enroll**
Account removed	Left the org	Re-invite and enroll
SCP modified	Edited `aws-guardrails-*` SCPs	**Re-register OU** / re-enable control
OU deleted / moved	Registered OU deleted in Organizations	**Repair landing zone**
Trusted access disabled	Organizations trusted access for CT off	**Repair landing zone**
Security Hub control drift	Service-managed standard changed	Re-enable / repair

- **Repair**: landing zone settings -> Repair (or `ResetLandingZone`).
- **Prevention**: change things **through Control Tower**, not directly in Organizations / Config / CloudFormation.
- StackSet failures in members: SCPs denying actions, disabled Regions, conflicting resources.

Integrations and boundaries:

- **CT doesn't enable GuardDuty** - make **Audit** the **delegated admin** with **auto-enable**; Security Hub can use the **CT service-managed standard**.
- **IAM Identity Center** managed by CT or by you.
- Account Factory and AFC blueprints are **Service Catalog** products.
- **AWS Backup** integration (central vaults, backup admin account).
- Organizations **SCP limits still apply** (5 per OU; CT combines its controls).
- **Organizations = primitives; Control Tower = opinionated automated best-practice setup** with controls, logging, SSO and account vending.

Part 10 - Exam traps

Trap	Truth
Build the baseline manually with Organizations + StackSets	**Control Tower** automates it
Change the home Region	**Not possible**
Block unused Regions	**Region deny control**
Disable a mandatory control	**Can't**
Detective control fixes issues	**Only reports**; add Config remediation
Block non-compliant CloudFormation resources before creation	**Proactive controls** (Hooks)
Preventive = Config rule	**Preventive = SCP / RCP**; detective = Config
Controls per individual account	**Per registered OU**
Enroll existing account fails	`AWSControlTowerExecution` **role** / Config recorder / SCPs
Bring a whole OU under CT	**Register the OU**
New landing zone version	**Update, then re-register OUs / update accounts**
Account moved in Organizations console	Drift; **update / re-enroll** or **repair**
Edited `aws-guardrails-*` SCP	Drift; **re-register / repair**
Self-service account requests	**Account Factory** (Service Catalog)
Automate account creation	`ProvisionProduct` on Account Factory (or AFT / CT APIs)
Standard VPC in new accounts	**Account Factory network configuration** (or blueprint / CfCT / AFT)
Terraform GitOps vending	**AFT**
Re-apply AFT customizations	`aft-invoke-customizations`
CloudFormation + SCPs auto-applied as code	**CfCT**
Simplest console customization	**AFC blueprints**
Automation after CT creates an account	`CreateManagedAccount` lifecycle event
CT enables GuardDuty	**No**; Audit as delegated admin
Where are the logs?	**Log Archive**
Where does security work?	**Audit** account
Control Tower cost	**Free**; pay for Config, CloudTrail, S3

Part 11 - Hands-on lab (about 90 minutes + waiting)

Use a **fresh test management account**: setup creates **2 extra accounts** and **Config in governed Regions** (small cost); decommissioning is slow. Keep governed Regions to **1 to 2**.

Lab 1 - Set up the landing zone

1. Control Tower -> **Set up landing zone**: home Region, at most one extra Region, **Region deny on**, Security + **Sandbox** OUs, new **Log Archive** + **Audit** emails, CloudTrail on, Identity Center managed by CT (30 to 60 minutes).
2. Explore **Organizations** (`aws-guardrails-*` SCPs), **Log Archive** buckets, **Audit** SNS topics, **Config aggregator**, **Identity Center** groups, **StackSets** `AWSControlTowerBP-*`.

Lab 2 - Controls

1. **Controls library**: filter by behaviour and guidance; a mandatory control has **no Disable**.
2. On **Sandbox** enable: preventive "Disallow creation of access keys for the root user"; detective "Detect whether public read access to S3 buckets is allowed"; a proactive S3 encryption control.
3. CLI: `aws controltower enable-control --control-identifier <control ARN> --target-identifier <OU ARN>`.

Lab 3 - Account Factory and lifecycle event

1. **Network configuration**: no VPC (or small CIDR, one Region).
2. **Create account** in **Sandbox** with an SSO user; watch the Service Catalog provisioned product (20 to 30 minutes).
3. EventBridge rule (management account, home Region) with an SNS email target:

```
{ "source": ["aws.controltower"],
  "detail-type": ["AWS Service Event via CloudTrail"],
  "detail": { "eventName": ["CreateManagedAccount"] } }
```

Create or update another account and receive the event.

Lab 4 - Test controls

Sign in through the **Identity Center portal**: a bucket in a **denied Region** is **blocked**; a CloudFormation stack with a non-compliant S3 bucket is **failed by the Hook**; a resource violating the detective control shows **non-compliant** on the dashboard.

Lab 5 - Drift and repair (optional)

In **Organizations**, move the account from Sandbox to the root: **drift**. Move it back and **update** it in Account Factory (or re-register the OU).

Lab 6 - Customizations

1. **AFC**: Account Factory customization settings (hub account + blueprints).
2. **CfCT** (optional): deploy the template, commit a `manifest.yaml` with a small StackSet (SSM parameter) and one SCP for Sandbox; watch CodePipeline; a new account gets it automatically.
3. **AFT** (reading): review the four repos in the docs / GitHub module.

Cleanup: disable elective controls, unmanage and close test accounts, decommission the landing zone only in a throwaway org (log buckets are retained).

Part 12 - Check yourself

Questions

1. Which services does Control Tower orchestrate, and does it cost money?
2. Which OU and accounts does the landing zone create, and what is each shared account for?
3. Can you change the home Region? How do you block unused Regions?
4. Preventive vs detective vs proactive - implementation and timing?
5. Mandatory, strongly recommended, elective? Can mandatory be disabled?
6. At what level are controls enabled?
7. A detective control shows non-compliant buckets - will CT fix them?
8. How do you bring an existing account under CT, and why might it fail?
9. Bring a whole existing OU under CT; what after a landing zone update?
10. How can account creation be automated and self-service with Account Factory?
11. What is AFT, where is it deployed, and its four repositories?
12. Re-apply AFT customizations to existing accounts?
13. What does CfCT deploy, from where, and how do new accounts get it automatically?
14. AFC vs CfCT vs AFT - when each?
15. Account moved in the Organizations console, dashboard shows drift - fix and prevention?

Answers

1. Organizations, Identity Center, CloudTrail, Config, Service Catalog, StackSets, S3, Security Hub; **CT is free**, pay for underlying services.
2. **Security OU** with **Log Archive** (central logs) and **Audit** (security roles, SNS, aggregator, delegated admin); optional **Sandbox** OU.
3. **No**; **Region deny control** for ungoverned Regions.
4. **Preventive** = SCP / RCP, blocks; **detective** = Config, reports after; **proactive** = CloudFormation Hook, checks before provisioning.
5. Mandatory always on, **can't disable**; strongly recommended = best practice; elective = optional.
6. **Registered OUs** (all their accounts).
7. **No**; add **Config remediation (SSM Automation)** or EventBridge + Lambda.
8. **Enroll** into a registered OU; needs `AWSControlTowerExecution`; fails on missing role, existing Config recorder, blocking SCPs.
9. **Register the OU**; after an update **re-register OUs / update accounts**.
10. Service Catalog product: `ProvisionProduct` or end-user access; AFT for Terraform.
11. Terraform **GitOps** vending in a **dedicated AFT account**; aft-account-request, aft-global-customizations, aft-account-customizations, aft-account-provisioning-customizations.
12. `aft-invoke-customizations` Step Functions.
13. **CloudFormation (StackSets) + SCPs / RCPs** from **manifest.yaml** (CodeCommit / S3) via **CodePipeline**; triggered by `CreateManagedAccount`.
14. **AFC** = simple console blueprints; **CfCT** = CloudFormation + SCPs as code; **AFT** = Terraform GitOps.
15. Move back / **update or re-enroll** (or **repair**); make changes **through Control Tower**.

Quick memory summary

- **Control Tower** = automated best-practice landing zone orchestrating Organizations, Identity Center, CloudTrail, Config, Service Catalog, StackSets, S3; **free**.
- **Landing zone**: **home Region (fixed)**, governed Regions + **Region deny**, **Security OU** with **Log Archive** + **Audit**, optional Sandbox, org trail, Config everywhere, SSO; **update -> re-register OUs**.
- **Controls**: **preventive (SCP / RCP)**, **detective (Config)**, **proactive (Hooks)**; **mandatory / strongly recommended / elective**; **per OU**; detective only reports.
- **Register OU / enroll account** (`AWSControlTowerExecution`); **drift** -> repair / re-register.
- **Account Factory** (Service Catalog, `ProvisionProduct`, VPC settings); **AFC blueprints**; **AFT** (Terraform, 4 repos, `aft-invoke-customizations`); **CfCT** (manifest.yaml, StackSets + SCPs, `CreateManagedAccount`); **lifecycle events** -> EventBridge.